SecureFalcon: Are We There Yet in Automated Software Vulnerability Detection With LLMs?

Mohamed Amine Ferrag, Ammar Battah, Norbert Tihanyi, *Member, IEEE*, Ridhi Jain, Diana Maimuţ, Fatima Alwahedi, Thierry Lestable, Narinderjit Singh Thandi, Abdechakour Mechri, Merouane Debbah, *Fellow, IEEE*, and Lucas C. Cordeiro

***Abstract*—Software vulnerabilities can cause numerous problems, including crashes, data loss, and security breaches. These issues greatly compromise quality and can negatively impact the market adoption of software applications and systems. Traditional bug-fixing methods, such as static analysis, often produce false positives. While bounded model checking, a form of Formal Verification (FV), can provide more accurate outcomes compared to static analyzers, it demands substantial resources and significantly hinders developer productivity. Can Machine Learning (ML) achieve accuracy comparable to FV methods and be used in popular instant code completion frameworks in near real-time? In this paper, we introduce `SecureFalcon`, an innovative model architecture with only 121 million parameters derived from the Falcon-40B model and explicitly tailored for classifying software vulnerabilities. To achieve the best performance, we trained our model using two datasets, namely the FormAI dataset and the FalconVulnDB. The FalconVulnDB is a combination of recent public datasets, namely the SySeVR framework, Draper VDISC, Bigvul, Diversevul, SARD Juliet, and ReVeal datasets. These datasets contain the top 25 most dangerous software weaknesses, such as CWE-119, CWE-120, CWE-476, CWE-122, CWE-190, CWE-121, CWE-78, CWE-787, CWE-20, and CWE-762. `SecureFalcon` achieves 94% accuracy in binary classification and up to 92% in multiclassification, with instant CPU inference times. It outperforms existing models such as BERT, RoBERTa, CodeBERT, and traditional ML algorithms, promising to push the boundaries of software vulnerability detection and instant code completion frameworks.**

***Index Terms*—FalconLLM, large language model, software security, security, generative pre-trained transformers.**

I. Introduction

As we are undoubtedly passing through a digital age, technology affects every aspect of our lives [1]. In such a context, vulnerability detection tools are essential safeguards in our continuously evolving digital landscape [2]. With the emergence of new technologies, the palette of cyber threats becomes wider and more sophisticated in terms of employed techniques. Vulnerability detection tools assist in scanning, probing, and inspecting software, identifying weaknesses that could serve as entry points for potential malicious users or attackers. While integral to software security measures, vulnerability detection tools encounter inherent limitations in their scope and efficiency. Such tools typically depend on known patterns and signatures derived from synthetic datasets, making them less effective for detecting bugs in real-world software [3]. Furthermore, the datasets used by these tools are often too small [4] or have a skewed distribution of vulnerable programs [5], hampering the accuracy of static analyzer tools. Although the advancements in deep learning (DL) seem promising for vulnerability detection [6], [7], [8], [9], their accuracy heavily relies on the data quality as well [5], [10]. Most of the popular datasets are either fully synthetic [11], [12], non-compilable [13], [14], [15], or have an unfair distribution of vulnerable vs non-vulnerable code [10], [14], [16]. Moreover, the labels associated with these datasets are subject to the detection method used. For instance, manual labeling is affected by human errors, whereas using static analysis tools to label the data can result in high false positives [17], [18]. While the DL-based approaches are often employed for quick inference, they may compromise the model's accuracy in detecting bugs. Formal Verification (FV) approaches such as Model Checking (MC) are preferred for verification in safety-critical systems [19]. Even though MC provides safety assurances for software systems [20], [21], they are often expensive, even for a small piece of code. Bounded Model Checking (BMC) [22], [23] improves the traditional MC techniques by restricting the exploration to a predefined bound or depth. To prove safety in BMC for programs, we must compute the completeness threshold (CT), which can be smaller than or equal to the maximum number of loop iterations occurring in the program. Although BMC offers performance improvement to some extent, it is still expensive.

Code completion tools are gaining popularity in software engineering, where rapid inference is essential. For example, tools such as GitHub Copilot[1] and Amazon Code Whisperer[2]

Received 31 May 2024; revised 26 December 2024; accepted 2 March 2025. Date of publication 5 March 2025; date of current version 18 April 2025. Recommended for acceptance by T. Menzies. *(Corresponding author: Mohamed Amine Ferrag.)*

Mohamed Amine Ferrag is with Guelma University, Guelma 24000, Algeria (e-mail: ferrag.mohamedamine@univ-guelma.dz).

Ammar Battah, Norbert Tihanyi, Ridhi Jain, Diana Maimut, Fatima Alwahedi, Thierry Lestable, Narinderjit Singh Thandi, and Abdechakour Mechri are with the Technology Innovation Institute, Abu Dhabi 9639, United Arab Emirates.

Merouane Debbah is with Khalifa University of Science and Technology, Abu Dhabi 127788, United Arab Emirates.

Lucas C. Cordeiro is with the University of Manchester, M13 9PL Manchester, U.K.

Digital Object Identifier 10.1109/TSE.2025.3548168

[1]https://github.com/features/copilot/

[2]https://aws.amazon.com/codewhisperer/

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

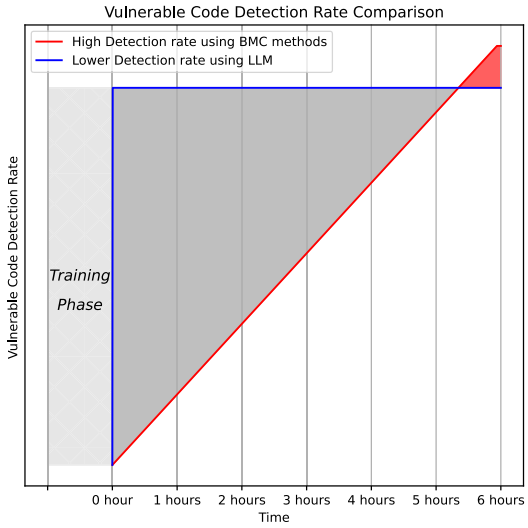


Fig. 1. Vulnerable code detection rate (BMC vs LLMs).

suggest code snippets based on contextual analysis and training data, which, according to recent studies, can also introduce vulnerabilities [24]. This raises a critical question: *Can we develop a model that detects vulnerabilities efficiently without the lengthy processing times associated with BMC methods while still maintaining high accuracy?* This is essentially a trade-off between accuracy and speed. BMC methods can achieve high accuracy by eliminating false positives with counterexamples and providing stack traces. Despite this, verifying the entire state space of even simple programs can take hours. This issue is illustrated in Fig. 1, where the red line represents the BMC method achieving high accuracy over several hours – an impractical timeframe for real-time code completion tools. Conversely, the blue line represents a Large Language Model (LLM), which, while not reaching the same accuracy as BMC methods, still predicts high reliability and can quickly identify software vulnerabilities.

The red-highlighted area represents the gap that can only be closed through rigorous mathematical formal verification [25]. However, overall, formal program verification is undecidable [26], [27]. We cannot devise a computational method to decide whether an arbitrary program is error-free. Much progress has been made in program verification. Nowadays, verification methods create abstractions to determine whether the program is error-free. Still, for an arbitrary program that includes unbounded memory usage, we cannot give 100% certainty that it is error-free due to the well-known *halting problem* [26]. However, to achieve high accuracy detection rate for vulnerable code almost instantly with a fine-tuned model would be a significant achievement. We aim to minimize this gap (red-highlighted area) as much as possible and dramatically reduce vulnerability detection times, making it practical for real-time code completion frameworks. While it is unrealistic to expect LLMs to match the results of BMC verification tools at this stage, there is promise in using ML techniques to swiftly identify the most vulnerable programs. Integrating a robust BMC tool with a pre-trained LLM can enhance the pace of vulnerability detection, benefiting the overall software development process. The success of a vulnerability detection model greatly depends on the quality and relevance of the dataset it uses. Thus, a dataset that provides a balanced distribution of vulnerable and non-vulnerable programs, mirrors real-world coding, and is labeled with a reliable method can enhance the model's accuracy. In contrast to most available C/C++ vulnerability datasets, the FormAI dataset [28] ① has high labeling accuracy as it is labeled by the Efficient SMT-based context-bounded model checker (ESBMC) [23], ② contains compilable programs, ③ evenly distributes vulnerable and neutral programs, and ④ is generated by LLM trained on real-world open source software, thus, mimicking the prominent developer errors and making it a highly suitable dataset for training a vulnerability detection model. The original contributions of our work are summarized as follows:

- We introduce `SecureFalcon`, a lightweight and innovative LLM model with only 121 million parameters, built on the foundational 40B-parameter *FalconLLM* architecture. This model offers significant enhancements and optimizations specifically tailored for security analysis. `SecureFalcon` was enhanced through fine-tuning using the FormAI dataset, a specialized collection designed to accurately classify vulnerabilities in C code samples. This process was supported by using the ESBMC tool, enhancing the model's detection capabilities.
- To address the limitations of formal verification tools, which are highly effective at detecting memory-related issues such as buffer overflows (CWE-121, CWE-122) and array bound violations (CWE-129) but are unable to identify vulnerabilities like OS command injection (CWE-78) or SQL injection (CWE-89), we created an aggregated dataset named *FalconVulnDB*. This dataset integrates resources from the SySeVR framework, Draper VDISC, Bigvul, Diversevul, SARD Juliet, and ReVeal datasets to enhance `SecureFalcon`'s training with a comprehensive collection of public datasets.
  These resources collectively cover examples of the top 25 most critical software weaknesses identified by the Common Weakness Enumeration (CWE), including but not limited to CWE-119, CWE-120, CWE-476, CWE-122, CWE-190, CWE-121, CWE-78, CWE-787, CWE-20, and CWE-762. The *FalconVulnDB* dataset was used to compensate for the lack of real project data in the FormAI dataset.
- In terms of performance, `SecureFalcon` showcases exceptional proficiency in binary classification, achieving an accuracy rate of 94% in identifying vulnerabilities in C/C++ code. Moreover, the model maintains a robust accuracy rate of 92% across diverse code samples in multi-classification, underscoring its effectiveness and reliability in vulnerability detection. With these results, we outperformed traditional ML algorithms like KNN, LR, NB, SVM, RRF, DT, and LDA by 11% (with RF achieving 81%), and existing LLM models like BERT, CodeBERT, and RoBERTa by 4% (with CodeBERT achieving 88%).

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

This advancement promises to push the boundaries of software vulnerability detection and instant code completion frameworks.

The rest of this paper is structured as follows. Section II outlines the main motivation behind our research. Section III overviews the research literature relevant to software vulnerability detection. Section IV discusses the methodology and approach employed to create `SecureFalcon`. Section VI presents the experimental setup and the evaluation of `SecureFalcon`'s performance. Finally, we conclude with Section VII summarizing the key concepts, findings, and contributions with their corresponding implications.

II. Motivation

As software development accelerates, so does the complexity of codebases, making identifying vulnerabilities a paramount concern. A tool that expeditiously pinpoints vulnerable code offers a proactive defense against potential exploits, minimizing the window of susceptibility and enhancing overall software security [29]. Rapid vulnerability detection not only safeguards against cyber threats but also instills confidence in software reliability, ensuring the delivery of secure and trustworthy applications within tight development schedules. Bounded Model Checking (BMC) is a formal verification technique used in computer science and software engineering to verify the correctness of critical hardware and software components within a finite number of steps. The BMC technique extracts the program code, the basis for generating a control-flow graph (CFG) [30]. In this CFG, each node represents either a deterministic or non-deterministic assignment or a conditional statement. The next step involves converting the CFG into a Static Single Assignment (SSA) form and transforming it into a State Transition System (STS). The resultant STS can subsequently be converted into an SMT (Satisfiability Modulo Theories) formula that can be understood by an SMT solver, such as CVC5 [31], Bitwuzla [32] or Z3 [33]. SMT solver tools can ascertain whether a counterexample exists for certain properties within a specified bound k. Formally written, given a program $\mathcal{P}$, consider its finite state transition system, $\mathcal{TS} = (S, R, I)$. Here, S is the set of states, $R \subseteq S \times S$ represents the set of transitions, and $(s_n, \cdots, s_m) \in I \subseteq S$ represents the set of initial states. A state $s \in S$ includes the program counter value, *pc*, and program variables. The initial state s_1 assigns the starting program location, and each transition $T = (s_i, s_{i+1}) \in R$ has a logical formula describing the constraints between states. In BMC, we define properties with logical formulas: $\phi(s)$ for safety/security properties and $\psi(s)$ for program termination. Notably, termination and error are exclusive: $\phi(s) \wedge \psi(s)$ is always unsatisfiable. A state is a deadlock if $T(s_i, s_{i+1}) \vee \phi(s)$ is unsatisfiable. The BMC problem, Δ_k can be expressed as:

$$\Delta_k = I(s_1) \wedge \bigwedge_{i=1}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=1}^{k} \neg\phi(s_i). \tag{1}$$

where I is the initial states set, and $T(s_i, s_{i+1})$ the transition relation of $\mathcal{TS}$. This formula captures $\mathcal{TS}$ executions of length k, and if it is satisfied, a state violates ϕ within the k limit. A counterexample, or trace, for a violated ϕ is a sequence of states $s_1, \ldots, s_k$. If Equation (1) is unsatisfiable, no error state exists within k steps, indicating no vulnerabilities in the program up to the k bound. Checking property violations using BMC techniques can be time-consuming even for relatively small programs due to lengthy loops that require unwinding or intricate function calls [23]. Consider Listing 1 where an arithmetic overflow occurs due to the nature of the nested loops and the accumulation of values in the *sum* variable. The code consists of two nested loops iterating from 0 to 999, 999. Within each iteration, the product of *i* * *j* is added to the sum variable. Since the *sum* variable accumulates these products (*sum* += *i* * *j;*), the *sum* eventually surpasses the maximum value that an int can represent. When this happens, it causes arithmetic overflow, which means the result is beyond the range represented by the data type, resulting in an incorrect value being stored in the *sum* variable.

Arithemtic overflow example

```
1  #include <stdio.h>
2   int main() {
3      int i, j;
4      int sum = 0;
5      for (i = 0; i < 1000000; ++i) {
6          for (j = 0; j < 1000000; ++j) {
7              sum += i * j;
8          }}
9      printf("Sum:%ld\n", sum);
10     return 0;
11 }
```

Listing 1. Arithmetic overflow on line number 7.

Verifying this program with BMC can take hours due to nested loops requiring careful unwinding. For example, applying the Efficient SMT-based Context-Bounded Model Checker (ESBMC) took more than seven hours using k-induction. In contrast, a suitably trained ML algorithm, such as a Large Language Model (LLM), can quickly pinpoint such issues. LLMs have several advantages over traditional formal methods like BMC: they can learn from vast amounts of code and vulnerability patterns, enabling them to generalize and identify errors efficiently. LLMs are also capable of handling diverse codebases and detecting vulnerabilities without the extensive manual setup required for BMC. This example underscores the necessity for a compact model that can swiftly identify common errors in C programs, enabling quicker detection across various scenarios. Additionally, LLMs can be integrated into code completion frameworks with near-instant inference time, providing real-time feedback to developers and significantly accelerating the development process. This makes LLMs a superior choice over BMC for many practical applications in software detection and vulnerability analysis.

III. Related Work

In recent research, LLMs have been employed for source code summarization [34], [35], which can aid in vulnerability

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

detection. Further, contrastive learning techniques have been applied to LLMs to improve vulnerability detection accuracy [36]. Similarly, transfer learning has been widely used in vulnerability detection and repair with LLMs [37], [38]. Researchers have significantly improved vulnerability detection performance by pre-training models on large code corpora and fine-tuning them on vulnerability-specific datasets. Neural machine translation techniques employed to translate code snippets into natural language aid in vulnerability detection [39], [40]. By translating code into human-readable descriptions, models can help developers understand code snippets' behavior and potential risks. SySeVR [41] uses DL to detect slice-level vulnerability by preserving semantic and syntactic knowledge about the vulnerabilities. Similarly, VulDeePecker [6] also extracts program slices to detect vulnerabilities. However, as these models are trained on semi-synthetic datasets, they fail to detect real-world vulnerabilities [10]. Devign [42] uses a Graph Neural Network (GNN) trained on manually labeled datasets for vulnerability identification. The Devigen dataset is constructed from the Linux kernel, QEMU, Wireshark, and FFmpeg, which generate around 58,000 graphs. However, the dataset only contains non-compilable functions. Several works have also used transformers' vulnerability detection due to their remarkable capabilities in understanding and processing natural language and code structures [9], [43], [44]. Their attention mechanisms enable them to capture complex relationships and patterns within text and programming languages. LLMs such as GPT-3 [45] have been explored for code security analysis and vulnerability detection. Researchers have used GPT to generate vulnerable code snippets and identify potential security flaws. CodeBERT [43], a pre-trained language model, has been applied to vulnerability detection in source code. By fine-tuning the model on labeled vulnerability data, researchers have achieved promising results in identifying vulnerabilities such as SQL injection, cross-site scripting (XSS), and buffer overflow [46], [47]. Transformer-based models, like BERT [48] and RoBERTa [44], have also been utilized for vulnerability detection in various software artifacts. Despite a recent surge in the application of DL for vulnerability detection [6], [49], [50], [41], a comprehensive solution with high confidence remains elusive [10], [51]. Most DL approaches suffer from four major issues: ① inadequate model, ② learning irrelevant features, ③ data duplication, ④ data imbalance [10].

To address these challenges, we employed a modern transformer model, the 40B parameter *FalconLLM*, which aids in comprehending semantic dependencies through extensive training (addressing problem ①). Falcon's architecture has demonstrated superior performance to GPT-3, achieving impressive results while utilizing only a portion of the training compute budget and requiring less computation at inference time. We fine-tuned the transformer model, leveraging the obtained understanding of natural language and learning directly from the source code (addressing problem ②). Further, we carefully preprocess the data to ensure no duplications or irrelevant features and minimize the class imbalance (addressing problems ③ and ④). We utilize only a configured portion of the Falcon-40B model for a light and compact model that fits the assigned task. Such a design choice stems from language models [52] and thorough experimentation, pushing us to take a modest approach to the scale of the model to combat overfitting concerns. We fine-tune the model on C/C++ code samples to be able to differentiate between vulnerable and non-vulnerable samples. The final model, which we named `SecureFalcon`, consists of only 121 million parameters.

IV. MODEL ARCHITECTURE

A. FalconLLM Model

The FalconLLM40B [53] is one of the best performing open-source models[3] that underwent an extensive training procedure on 384 GPUs (A100 40GB). The model's training procedure incorporated a 3D parallelism strategy that involved tensor parallelism of 8 (TP=8), pipeline parallelism of 4 (PP=4), and data parallelism of 12 (DP=12). This approach was used in conjunction with ZeRO (Zero Redundancy Optimizer) to enhance the efficiency of the training procedure.

The training hyperparameters used for FALCONLLM40B were specifically selected to optimize the model's learning process. The precision of the model was set to bfloat16 to balance computational efficiency and numerical precision. The *AdamW*! [54] optimizer was chosen for its proven ability to achieve good results in less time. The learning rate was set at 1.85e-4 during the warm-up phase involving 4 billion tokens, followed by a cosine decay to 1.85e-5, which allows the model to converge more efficiently. The weight decay was set at 1e-1 to prevent overfitting, while Z-loss was set at 1e-4 to minimize the discrepancy between the model's predictions and the true values. Finally, the batch size was fixed at 1152 with a 100 billion token ramp-up to maximize computational throughput and stabilize the learning process.

B. SecureFalcon Model Architecture

The *SecureFalcon* model, derived from the 40B parameter FalconLLM, includes components shown in Fig. 2, with `out_features=12` for multi-class classification and `out_features=2` for binary classification.

The architecture comprises four primary components: *Word Embeddings*, *Encoder Layers*, *Final Layer Normalization*, and the *Scoring Layer*.

1) Word Embeddings: Word Embeddings serve as the initial transformation layer in the language model. This layer transforms discrete words into dense, continuous vectors, which encapsulate semantic and syntactic information of the words [55]. Their dimension is 768, and the model is trained with a vocabulary size 65024.

Let's denote the i-th word in an input sequence as $input[i]$. The corresponding word embedding, e_i, is a row vector obtained from the embedding matrix E, which can be expressed as:

$$e_i = E(input)[i, :] \tag{2}$$

[3]https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard - 10 July 2023

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

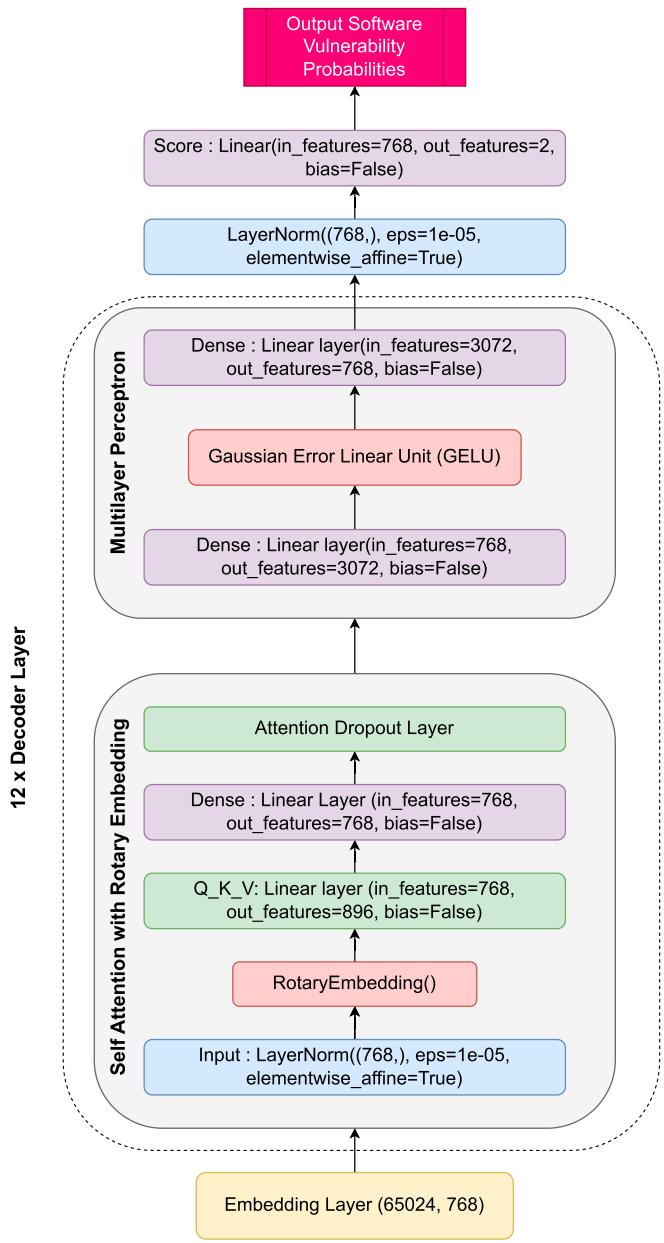


Fig. 2. *SecureFalcon* model architecture.

Here, E represents the embedding matrix with dimensions *vocabulary size* $\times$ *embedding dimension*, which in this case is 65024×768. Each row in E corresponds to the vector representation of a word in the vocabulary.

Therefore, an input sequence $input$ of length n is transformed into a sequence of word vectors e of dimension $n \times 768$. This transformation can be depicted as:

$$e = [e_1, e_2, \ldots, e_n], \text{where } e_i = E(input)[i, :] \tag{3}$$

These word embeddings, e_i, are fed into subsequent layers in the model. They encapsulate rich information about the semantics and syntactic roles of the words and their context, providing a dense representation that assists in better understanding and generating language.

2) Decoder Layers: These constitute the main portion of the language model and comprise four stacked transformer layers [56]. Each layer includes the following components:

- *Layer Normalization*: This regularization technique standardizes the inputs across the feature dimension, making the model more stable and faster to train [57]. The parameters for this layer are 768-dimensional vectors. It's computed using the formula:

$$\hat{x} = \frac{x - \mu}{\sigma} \tag{4}$$

where x is the input, μ is the mean, σ is the standard deviation, and $\hat{x}$ is the normalized output. This operation is applied to each feature independently.

- *Self-Attention with Rotary Position Embedding (RoPE)*: Recent advancements in position encoding have shown its effectiveness within the Transformer architecture, offering valuable supervision for dependency modeling between elements at different sequence positions. In the pursuit of integrating positional information into the learning process of Transformer-based language models, a novel method, termed Rotary Position Embedding (RoPE), has been proposed by Su et al. [58]. RoPE uniquely encodes the absolute position using a rotation matrix while incorporating explicit relative position dependency in the self-attention formulation. RoPE's implementation involves applying the rotation to the query (Q) and key (K) vectors:

$$Q', K' = \text{rotate}(Q, K, \text{RoPE}) \tag{5}$$

Where rotate is a function that applies RoPE to the vectors. The self-attention function is subsequently defined as:

$$\text{Att}(Q', K', V) = \text{softmax}\left(\frac{Q'K'^T}{\sqrt{d_k}}\right)V \tag{6}$$

Where d_k denotes the dimension of the key vectors. The softmax function ensures that the aggregate weight of different values equals 1. Division by $\sqrt{d_k}$ is a scaling factor to maintain gradient stability during optimization. RoPE exhibits several beneficial properties, such as sequence length flexibility, a decay in inter-token dependency with increasing relative distances, and the capability of endowing linear self-attention with relative position encoding.

- *MLP (Multilayer Perceptron)*: The MLP is a type of neural network that comprises a minimum of three layers of nodes: an input layer, one or more hidden layers, and an output layer [59]. Each layer is fully connected to the subsequent layer, signifying that every node in one layer is connected with every node in the following layer. In this case, the MLP includes an input layer. This hidden layer uses a Gaussian Error Linear Unit (GELU) activation function [60] to introduce non-linearity and an output layer. This non-linearity allows the model to capture and learn complex patterns in the data. The MLP can be represented as:

$$MLP(x) = Lin_{\text{out}}(GELU(Lin_{\text{hidden}}(x))) \tag{7}$$

In this equation, Lin_{hidden} is the linear transformation corresponding to the hidden layer, $GELU$ represents the

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

Gaussian Error Linear Unit activation function, and Lin_{out} denotes the linear transformation of the output layer. The variable x represents the input to the MLP.

Each of the above components is essential to the functionality of the decoder layer. The layer normalization stabilizes the inputs to the self-attention and MLP components, the self-attention module allows the model to consider different parts of the input when generating each word, and the MLP provides the additional representative capacity to the model.

3) Final Layer Normalization: The output from the last decoder layer undergoes an additional layer normalization operation to standardize the outputs before the final linear transformation and softmax operation. This operation follows the same mathematical principles as the layer normalization in the decoder layers. The parameters for this layer are also 768-dimensional vectors. This layer maintains a consistent distribution of activations and gradients across the network, improving model performance. In the context of language models, this helps preserve the quality of the generated text. The operation of the final layer normalization can be represented as:

$$\hat{y} = \frac{y - \mu}{\sigma} \tag{8}$$

where y is the input to the final layer normalization, μ is the mean, σ is the standard deviation, and $\hat{y}$ is the normalized output. Similar to the layer normalization in the decoder layers, this operation is applied to each feature independently. After the final layer normalization, the 768-dimensional vectors are passed into the final linear layer and softmax function to generate the output probabilities for each word in the vocabulary.

4) Scoring Layer: The scoring stage involves a linear layer that generates vulnerability scores for the input software code [61]. This layer is engineered to transform the normalized decoder output of 768 dimensions into a 2-dimensional vector, aligning with the vulnerability classes ("vulnerable" and "not vulnerable"). Similarly, the output features are expanded to 12 dimensions for multiclass classification.

Let's represent the decoder output, which has a dimension of 768, as d. In a binary classification scenario, we denote the weight matrix as W with dimensions 768x2, and for multiclass classification, as W with dimensions 768x12. Additionally, we denote the bias vector as b. Then, the score can be computed with the linear transformation as:

$$\text{Score} = W^T d + b \tag{9}$$

where $W^T d$ represents the matrix multiplication of the transpose of the weight matrix W and the decoder output vector d. The score vector is then passed through a sigmoid function to convert the scores into probabilities. The sigmoid function can be defined as:

$$P(\text{class}_i) = \frac{1}{1 + e^{-\text{Score}_i}} \tag{10}$$

where $P(\text{class}_i)$ is the probability of the i-th class, and Score_i is the score for the i-th class. The final output represents the model's prediction of the vulnerability status of the input code.

The model outputs a score for each category of vulnerabilities. The class with the highest score is considered the model's prediction. Being fine-tuned from the *FalconLLM* model, this architecture has proven effective for software vulnerability detection by capturing the complex syntax and semantics of programming languages, which will be demonstrated in the Experimental Evaluation Section.

TABLE I
DATA DISTRIBUTION FORMAI DATASET

Class	Samples	Before pre-processing		After pre-processing	
		Training	Testing	Training	Testing
0	45275	40747	4528	40745	4528
1	197800	178020	19780	55374	15533

0: NOT VULNERABLE, 1: VULNERABLE

V. DATASETS FOR FINE-TUNING SECUREFALCON

To develop a robust model, the choice of dataset is essential. During the training phase, we utilized two datasets: the FormAI dataset [28] and FalconVulnDB, an aggregated dataset we created for fine-tuning `SecureFalcon`, compiled from all relevant datasets found in the literature.

A. FormAI Dataset

SecureFalcon uses the FormAI dataset [28] for fine-tuning the model. The FormAI dataset includes $112,000$ compilable C code snippets created using the GPT-3.5-turbo model through a dynamic zero-shot prompting method. Since this GPT model from OpenAI[4] is trained using real-world open-source software repositories, the code it generates closely mimics real-world code behavior. The produced C samples are subsequently verified using ESBMC [62].

ESBMC is set with a verification timeout of 30 seconds per sample. According to the 2023 SV-COMP results[5], this verifier has successfully addressed the most significant number of verification tasks. The generated C samples are classified into three categories: verification successful, verification unknown, and verification failed. The verification failed class is further divided into specific vulnerability types: arithmetic overflow, buffer overflow, array bounds violation, NULL pointer dereference, division by zero, and others. The dataset provides insights into the distribution of vulnerabilities and serves as a valuable resource for vulnerability analysis and testing.

In the FormAI dataset, the classification of C programs based on vulnerability reveals that out of a total of $112,000$ programs, the verification process was performed on $106,139$ programs. Among these, $57,389$ unique programs were classified as vulnerable, resulting in $197,800$ vulnerabilities (vulnerable functions). Table I provides an overview of the number of samples of the FormAI dataset in two classes, 'NOT VULNERABLE' (Class 0) and 'VULNERABLE' (Class 1), before and after a data pre-processing stage.

Initially, the 'NOT VULNERABLE' class had 40,747 training and $4,528$ testing samples, out of a total of $45,275$, with the figures remaining almost the same post-pre-processing.

[4]https://openai.com/

[5]https://sv-comp.sosy-lab.org/2023/results/results-verified/quantilePlot-Overall.svg

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STATISTICS OF FORMAI DATASET'S DISTRIBUTION

Statistic	Value
Count	243,075
Mean	271.69
Std	162.25
Min	9
25%	160
50%	235
75%	343
Max	2,059

The 'VULNERABLE' class starts with significantly more samples, $178,020$ in training, and $19,780$ in testing out of $197,800$. However, training samples reduce to $55,374$ after pre-processing, and testing samples decrease to $15,533$. Table II presents the key statistical measurements of the FormAI dataset. The dataset comprises $243,075$ observations in total. The average or mean value of these observations is 271.69, with a standard deviation (Std) of 162.25, indicating the spread or variability of the dataset. The dataset's minimum (Min) value is 9, while the maximum (Max) value observed is $2,059$. The dataset's quartile distribution is also presented, with the 25th percentile (25%) at 160, the median or the 50th percentile (50%) at 235, and the 75th percentile (75%) at 343. These figures help describe the dataset's central tendency, dispersion, and distribution shape.

In addition, the vulnerabilities in the dataset are associated with registered Common Weakness Enumeration (CWE) identifiers, where each vulnerability class may map to multiple CWEs. The dataset comprises nine categories that address the most common vulnerabilities across 42 unique CWEs, including an "others" category containing various other instances with fewer occurrences.

1. Arithmetic overflow
2. Buffer overflow on `scanf()/fscanf()`
3. Array bounds violated
4. Dereference failure: NULL pointer
5. Dereference failure: forgotten memory
6. Dereference failure: invalid pointer
7. Dereference failure: array bounds violated
8. Division by zero
9. Other vulnerabilities

Each category is associated with specific CWE numbers that capture the weaknesses leading to those vulnerabilities. For example, arithmetic overflow is associated with CWE-190, CWE-191, CWE-754, CWE-680, and CWE-681. The dictionary of the CWE-IDs associated with vulnerability types is maintained by the MITRE Corporation[6].

The pre-processing steps performed on the FormAI dataset are crucial for preparing the data before feeding it into a *FalconLLM* model. The pre-processing includes removing header information and cleaning the text by removing HTML tags, links, and email addresses. These steps help standardize the text data and eliminate noise or irrelevant information hindering the subsequent analysis. In addition, we calculate the number of words in each text entry and add a new column for the word count. This new column calculates the maximum length of an input sequence that the tokenizer will accept. We convert the categorical 'label' column to a numerical representation using label encoding. Generally, this pre-processing stage guarantees that the dataset is sanitized, standardized, and prepared for the fine-tuning process of *FalconLLM* using the FormAI dataset. Fig. 3 presents the top 10 most frequent vulnerability categories in the FormAI dataset after the pre-processing. Specifically, we find that *'buffer overflow on scanf'* is the most prevalent vulnerability, recorded 29,843 times. This is followed by *'dereference failure: array bounds violated'* and *'dereference failure: NULL pointer'* with 6,187 and 5,436 instances, respectively. Other notable vulnerabilities include *'dereference failure: invalid pointer'* and several types of *'arithmetic overflow'* errors, such as on *'sub'*, *'add'*, *'floating-point ieee_mul'*, *'floating-point ieee_div'*, and *'mul'*, with counts ranging from 2,959 to 580. *'Buffer overflow on fscanf'* also emerges as a significant vulnerability with 625 instances.

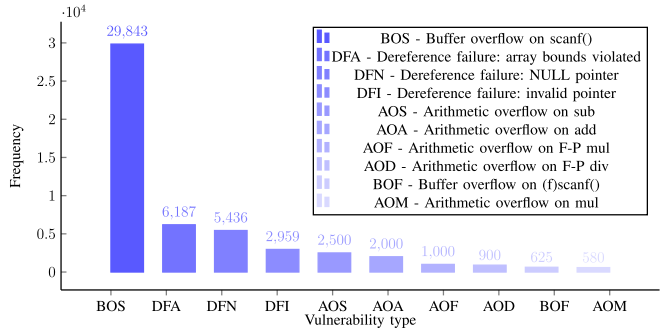


Fig. 3. Top 10 most frequent vulnerabilities categories in the FormAI dataset.

B. Limitations of AI-Generated Training Data

While including the FormAI dataset allowed us to scale our training corpus with synthetically generated samples rapidly, we recognize that this dataset alone may not fully capture the complexities of real-world code. AI-generated code often lacks the historical context, unconventional patterns, and domain-specific styles in human-written codebases. To mitigate this limitation, we integrated the SySeVR framework, Draper VDISC, Bigvul, Diversevul, SARD Juliet, and ReVeal datasets into a consolidated resource called *FalconVulnDB*. This aggregated dataset encompasses examples of the top 25 most critical software weaknesses (CWE), such as CWE-119, CWE-120, CWE-476, CWE-122, CWE-190, CWE-121, CWE-78, CWE-787, CWE-20, and CWE-762, drawn from public, human-curated sources.

By leveraging *FalconVulnDB*, we aim to compensate for the scarcity of genuine project data in the FormAI corpus. Unlike AI-generated code, these publicly available datasets capture various coding habits, legacy structures, and context-driven vulnerabilities. The combination of AI-synthesized and human-sourced data fosters a more balanced training

[6]https://cwe.mitre.org/

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

environment, encouraging our model to learn from the syntactic consistency of machine-generated samples and the gritty realism of authentic software repositories.

C. FalconVulnDB Dataset

We also fine-tuned `SecureFalcon` using datasets cited in the literature, specifically: the SySeVR framework [41], Draper VDISC [15], Bigvul [14], Diversevul [16], SARD Juliet [12], andReVeal [10]. Each dataset consisted of varying elements: the first included 1,591 programs, the second held 1.27 million functions, the third contained over 264,000 functions, the fourth had more than 348,000 functions, the fifth comprised over 64,000 test cases, and the sixth featured more than 22,000 functions. Except for data from Juliet, the data is scraped from open-source projects. Test cases in Juliet are vulnerable by design and have their associated patch with them. Furthermore, we obfuscate the function and variable names as they were explicit and would affect the model's training. As for other datasets, limited pre-processing was done on them, such as function extraction, dealing with line breaks, removing comments, CWE mapping, and unifying their features. While several datasets have CWE mapping, some are missing CWE and have a CVE instead. In the case of ReVeal, it has no CWEs and is just labeled as vulnerable or not. At the same time, the Draper dataset is the only multi-labeled dataset.

The amalgamation of the dataset resulted in the outcome presented in Table III. The dataset initially contained 1.7 million samples. However, there is an overlap between the different datasets, which we remove after the pre-processing. We employ more than 750 000 samples to train the model, with 20% being a test set. The following CWEs[7] are included in FalconVulnDB.

- **CWE-20: Improper Input Validation** - Occurs when software does not validate or improperly validates input, affecting a program's control or data flow. This can lead to unauthorized access, denial of service, or privilege escalation.
- **CWE-78: OS Command Injection** - An application allows the execution of arbitrary OS commands due to inadequate input validation, which can result in a complete system takeover.
- **CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer** - A buffer overflow occurs when a program operates on more data than the size of its memory buffer. It can allow arbitrary code execution, control flow alteration, or system crash.
- **CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')** - A specific instance of buffer overflow caused by buffer copy operations without adequate size checks of the input.
- **CWE-121: Stack-based Buffer Overflow** - Occurs in stack memory, potentially leading to arbitrary code execution or manipulation of program execution flow by overwriting critical data.
- **CWE-122: Heap-based Buffer Overflow** Similar to stack-based but occurs in heap memory, leading to data corruption or unexpected behavior through manipulated pointers.
- **CWE-190: Integer Overflow or Wraparound** - Happens when an integer operation produces a value too large to be held by the integer type, causing the value to wrap and create unintended values, leading to errors or vulnerabilities.
- **CWE-476: NULL Pointer Dereference** - It occurs when a program dereferences a pointer, which it expects to be valid but is NULL, leading to crashes or code execution.
- **CWE-762: Mismatched Memory Management Routines** - Arises when memory is allocated and deallocated with different routines, potentially leading to heap corruption or crashes.
- **CWE-787: Out-of-bounds Write** - It happens when software writes data outside the intended buffer boundaries, leading to data corruption, crashes, or code execution vulnerabilities.

[7]https://cwe.mitre.org/top25/archive/2023/2023_kev_list.html

TABLE III
DATA DISTRIBUTION OF FALCONVULNDB

Class	Samples	Before pre-processing		After pre-processing	
		Training	Testing	Training	Testing
0	1,611,613	1,289,217	322,396	497,116	124,679
1	139,616	111,766	27,850	116,404	28,702

0: NOT VULNERABLE, 1: VULNERABLE

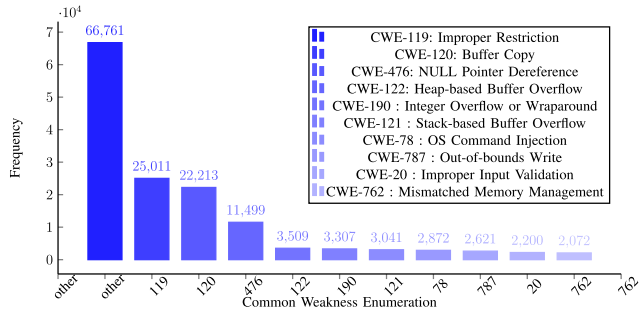


Fig. 4. Top 11 most frequent vulnerability categories in FalconVulnDB.

The most common vulnerability categories in FalconVulnDB are displayed in Fig. 4.

D. Data Integration, Standardization, and Bias Mitigation

We acknowledge the known data quality issues associated with widely used datasets like BigVul, as highlighted in prior studies such as LineVul [63]. To ensure the reliability of our results, we implemented a rigorous pre-processing pipeline that included deduplication to eliminate redundant records, validation of vulnerability labels by cross-referencing with publicly available vulnerability descriptions, and removing incomplete or ambiguous entries.

We employed a structured integration and standardization process to ensure consistency, reliability, and transparency in combining these heterogeneous sources. First, we established a unified schema that standardized fields such as vulnerability labels, CWE tags, and severity metrics. For datasets that provided direct CWE mappings, we incorporated them as-is; for those offering only CVE references or binary vulnerability labels (e.g., ReVeal), we cross-checked these identifiers against authoritative databases (e.g., NVD) to map or infer the most likely CWE categories whenever possible. For instance, if a function marked as vulnerable in ReVeal referenced a CVE later confirmed by NVD as related to a buffer overflow vulnerability, we annotated that sample with the corresponding CWE (e.g., CWE-120). When no reliable CWE classification could be assigned, we preserved the binary vulnerability label to maintain dataset diversity.

We also addressed overlaps and conflicts among datasets. In one example, the same function appeared in both Big-Vul and SySeVR, but SySeVR indicated it had been patched, while Big-Vul still listed it as vulnerable. We adopted a hierarchical decision approach in such cases: we consulted NVD and official vendor advisories to verify the patch status. If confirmed, the function was recorded as patched and no longer considered an

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

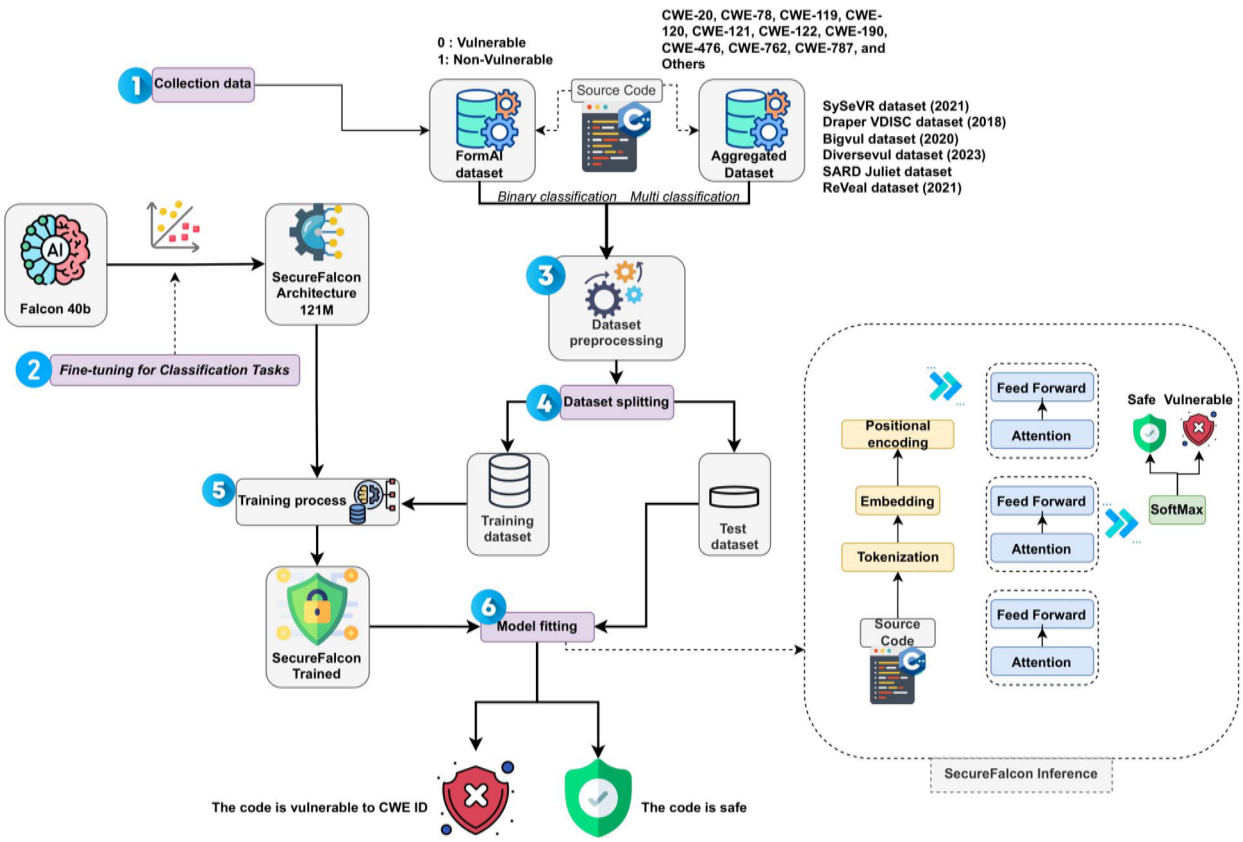


Fig. 5. Performance evaluation steps of *SecureFalcon* model.

active vulnerability. Another example involved duplicates in which multiple datasets reported the same CVE but provided slightly different severity scores. Here, we normalized the severity by referencing a standard scale (e.g., CVSS) and took the median of reported severity metrics, ensuring a more robust and standardized representation of the vulnerability's impact.

To reduce bias, we examined the data distribution across various dimensions—such as vulnerability type, programming language, and software domain—and identified areas of overrepresentation. For example, if stack-based buffer overflows (CWE-121) were disproportionately frequent due to their abundance in a particular dataset, we incorporated more diverse examples from other sources or filtered out duplicates to achieve a more balanced distribution. Additionally, we validated portions of FalconVulnDB against established benchmarks like NVD. For example, if our integrated dataset suggested null pointer dereferences (CWE-476) were exceedingly rare compared to NVD distributions, we revisited our filtering and inclusion criteria to better align with real-world data.

VI. Performance Evaluation

Performance evaluation of `SecureFalcon` model for software vulnerability detection involves a series of steps, as presented in Fig. 5. First, we collect a large and diverse dataset of software code from the FormAI and the aggregated datasets (FalconVulnDB), where instances of vulnerable and non-vulnerable code are correctly labeled. Once we have the datasets, we pre-process them by transforming the raw source code into a format suitable for the `SecureFalcon` model, tokenizing the code into appropriate separate sub-strings. Next, we initiate the `SecureFalcon` model with the desired architecture using the Transformers library. Then, we define the training and evaluation settings, which involve setting various hyperparameters like learning rate, batch size, and number of epochs. We start the fine-tuning process, where the model will learn from the labeled dataset to differentiate between vulnerable and non-vulnerable code. We evaluate the model regularly during training to track its progress and adjust hyperparameters when necessary. After training the model, we test it on separate data amalgamated from the different datasets to verify its effectiveness. Lastly, after fine-tuning, the resulting model should be able to predict whether a given piece of code is vulnerable or not.

A. Experimental Setup

Table IV outlines the experimental setup and parameters used for fine-tuning `SecureFalcon` for software security.

The `SecureFalcon` model features 121 million parameters, a hidden size of 768, twelve hidden layers, and twelve attention heads. This model employs an intermediate size of 3072 and supports up to 514 position embeddings. Tokenization was performed using a left-padding approach, with a padding token ID, BOS (beginning of sentence) token ID, and EOS (end of sentence) token ID all set to 11. The tokenizer settings

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
Configuration of SECUREFALCON

Parameter	Value
Pretrained model ID	FalconLLM 40B
Number of parameters	121M
Hidden size	768
Number of hidden layers	12
Number of attention heads	12
Intermediate-size	3072
Maximum position embeddings	514
Number of labels	2/12
Tokenizer padding side	Left
Padding token ID	11
BOS token ID	11
EOS token ID	11
Maximum length of tokens	2048
Tokenizer truncation	True
Tokenizer padding	True
Return tensor format	PyTorch tensors ('pt')
Return token type IDs	False
Return attention mask	True
Batch size	256
GPU	32 A100 40GB
Optimizer	AdamW
AdamW Learning Rate (LR)	2e-2 and 2e-5
AdamW Epsilon	1e-8
Number of training epochs	10
Early stopping	Enabled (patience=3)
Random seed value	42
Maximum gradient norm	1.0
Loss computation	Cross-entropy loss
Hidden activation function	"gelu"
Initializer range	0.02
Layer norm epsilon	1e-5
Attention probs dropout prob	0.1
Hidden dropout prob	0.1
Torch_dtype	float32
Transformers_version	4.30.2
SMDDP_version	2.1.0
Pytorch_version	2.1.0
Python_version	3.10

enforced truncation and padding, producing PyTorch tensor outputs and including an attention mask, while token type IDs were not returned. We set the maximum token length to 2048. For training, batch sizes were set to 256, utilizing 32 Nvidia A100 40GB GPUs to ensure efficient processing. We employed the AdamW optimizer with learning rates set at 2e-2 and 2e-5 and an epsilon value of 1e-8. The training was carried out over ten epochs to prevent overfitting, with early stopping enabled after three epochs without improvement. For loss computation, we utilized cross-entropy loss. The model was evaluated based on accuracy, precision, recall, and the F1 score, with metrics averaged using the 'micro' method to reflect the contribution of each instance to the overall metric. The additional configuration included setting the hidden activation function to GELU, an initializer range of 0.02, and a layer norm epsilon of 1e-5. The dropout probabilities for attention and hidden layers were set to 0.1. The models were run under a float32 torch data type setting, ensuring compatibility and optimal performance on the specified transformer library version 4.30.2.

TABLE V
CLASSIFICATION REPORT OF SECUREFALCON 121M WITH LR = 2E-5 USING FORMAI DATASET

	Precision	Recall	F1-Score	Support
0	0.89	0.84	0.86	4528
1	0.95	0.97	0.96	15533
Accuracy		0.94		
Macro avg	0.92	0.90	0.91	20061
Weighted avg	0.94	0.94	0.94	20061

0: NOT VULNERABLE, 1: VULNERABLE.

TABLE VI
CLASSIFICATION REPORT OF SECUREFALCON 121M WITH LR = 2E-2 USING FORMAI DATASET

	Precision	Recall	F1-Score	Support
0	0.67	0.80	0.73	4528
1	0.94	0.88	0.91	15533
Accuracy		0.87		
Macro avg	0.80	0.84	0.82	20061
Weighted avg	0.88	0.87	0.87	20061

0: NOT VULNERABLE, 1: VULNERABLE.

TABLE VII
TRAINING AND VALIDATION ACCURACY AND LOSS ACROSS EPOCHS OF SECUREFALCON 121M WITH LR = 2E-5 USING FORMAI DATASET

	Training		Validation	
Epoch	Accuracy	Loss	Accuracy	Loss
1	0.82	0.39	0.88	0.27
2	0.87	0.29	0.89	0.25
3	0.89	0.25	0.90	0.23
4	0.91	0.21	0.91	0.21
5	0.93	0.17	0.93	0.19
6	0.95	0.12	0.93	0.19
7	0.97	0.09	0.94	0.19

B. Experimental Results

Tables V and VI present the classification report of `SecureFalcon`-121M with LR = 2e-5 and LR = 2e-2. With an LR of 2e-5, the model yields the highest accuracy of 0.94, supported by a high precision, recall, and F1-score for both classes (0.89, 0.84, 0.86 for 'NOT VULNERABLE' and 0.95, 0.97, 0.96 for 'VULNERABLE') (see Tables V)

This performance considerably drops when the learning rate is increased to 2e-2. Although the precision for 'VULNERABLE' remains high (0.94), the 'NOT VULNERABLE' metrics take a substantial hit, decreasing overall accuracy to 0.87.

Tables VII, VIII, IX present the training and validation accuracy and loss over several epochs for `SecureFalcon` model with differing configurations. With a Learning Rate (LR) of 2e-5 (Table VII), accuracy consistently increases, and loss decreases over the epochs for both training and validation. However, with an LR of 2e-2 (Table VIII), lower accuracy and higher loss indicate that a larger learning rate could lead to sub-optimal learning.

Fig. 6 illustrates the confusion matrix for the binary classification performed by `SecureFalcon`. From the provided confusion matrices, we can make some observations regarding

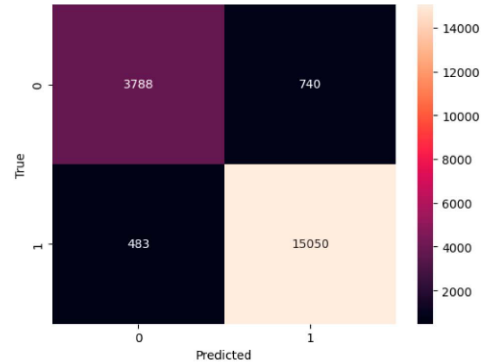
Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
TRAINING AND VALIDATION ACCURACY AND LOSS ACROSS EPOCHS OF SECUREFALCON 121M WITH LR = 2E-2 USING FORMAI DATASET

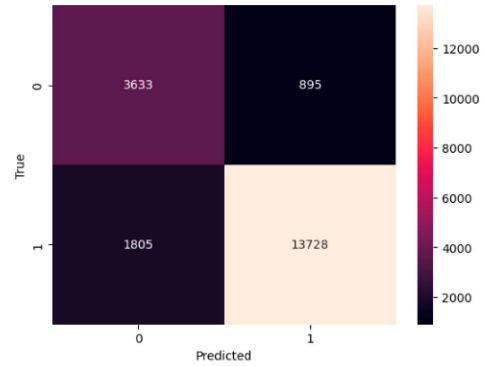
	Training		Validation	
Epoch	Accuracy	Loss	Accuracy	Loss
1	0.70	0.61	0.82	0.43
2	0.78	0.48	0.80	0.42
3	0.80	0.44	0.85	0.34
4	0.83	0.38	0.81	0.41
5	0.82	0.40	0.86	0.31
6	0.84	0.36	0.86	0.30
7	0.85	0.34	0.86	0.30
8	0.85	0.34	0.87	0.30

TABLE IX
TRAINING AND VALIDATION ACCURACY AND LOSS ACROSS EPOCHS OF SECUREFALCON 121M WITH LR = 2E-5 USING FALCONVULNDB DATASET

	Training		Validation	
Epoch	Accuracy	Loss	Accuracy	Loss
1	0.91	0.30	0.91	0.31
2	0.92	0.26	0.91	0.32



(a) SecureFalcon with LR = 2e-5



(b) SecureFalcon with LR = 2e-2

Fig. 6. Confusion matrix of SecureFalcon 121M classification using FormAI dataset.

the performance of the SecureFalcon models with different configurations. A decrease in performance is observed when the learning rate is raised to 2e-2. The number of false negatives drastically increased from 483 to 1805, with false positives rising from 740 to 895. Correspondingly, the true positives and negatives decreased from 15050 to 13728 and from 3788 to 3633, respectively. This shows that a larger learning rate, in this case, led to more misclassifications, indicating an over-optimistic learning process that possibly led to overfitting or instability during training. This is due to the high learning rate contributing to more drastic changes in weight, leading to instability or overfitting.

The multi-classification report of the SecureFalcon model, as presented in Table X, exhibits a comprehensive evaluation of the model's performance across various Common Weakness Enumerations (CWEs) on FalconVulnDB. We conducted several experiments to determine the model's rationale for the classification. One of the interpretability tools[8] used was a library highlighting the distribution of attention, indicating some discrepancies in our model results. To enhance the knowledge of the model of the syntactic and lexical nature of the programming language, we embed the tokens shown in Table XI. The tokens are keywords, punctuation, and API calls defined in the C/C++ manual and its extended libraries[9,10], an approach similar to work [9]. Testing with the new tokens included in the fine-tuning, in turn, updating the embedding layer has yielded higher precision and recall. This is due to maintaining the syntax of the pre-defined tokens rather than going through the subword tokenization process. As such, the meaning of these essential pre-defined tokens in the source code is preserved.

TABLE X
CLASSIFICATION REPORT OF SECUREFALCON 121M WITH LR = 2E-5 USING FALCONVULNDB DATASET

	Precision	Recall	F1-Score	Support
Not-Vulnerable	0.93	0.97	0.95	124355
CWE-20	0.46	0.14	0.22	440
CWE-78	1.00	0.98	0.99	574
CWE-119	0.75	0.75	0.75	5002
CWE-120	0.63	0.84	0.72	4443
CWE-121	0.99	0.99	0.99	608
CWE-122	1.00	0.99	0.99	702
CWE-190	0.97	0.78	0.87	662
CWE-476	0.64	0.45	0.53	2300
CWE-762	1.00	1.00	1.00	414
CWE-787	0.27	0.26	0.26	524
Other	0.89	0.54	0.67	13352
Accuracy		0.92		
Macro Avg	0.79	0.72	0.74	153376
Weighted Avg	0.91	0.91	0.90	153376

The model demonstrates high precision and recall for most categories, particularly excelling in identifying non-vulnerable instances with a precision of 0.93, recall of 0.97, and an

[8]https://github.com/cdpierse/transformers-interpret (accessed 9 May 2024).
[9]https://www.gnu.org/software/libc/manual/pdf/libc.pdf (accessed 15 May 2024).
[10]https://learn.microsoft.com/en-us/cpp/cpp/cpp-language-reference (accessed 15 May 2024).

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
LIST OF SPECIAL TOKENS ADDED IN THE FALCON40B TOKENIZER

Tokens	Count	Examples
Punctuation	72	!=, ++, =
Keywords	123	char, const, continue
API calls	394	malloc, strncpy, atoi

F1-score of 0.95, indicating robustness in distinguishing non-vulnerable cases. Remarkably, it achieves perfect or near-perfect performance in identifying CWE-78, CWE-121, CWE-122, and CWE-762 vulnerabilities, showcasing its effectiveness in detecting specific types of vulnerabilities with high accuracy. However, the model shows limitations in recognizing certain vulnerabilities, notably CWE-20 and CWE-787, with notably lower precision and recall values, suggesting areas for improvement in future iterations. The low F1-score for CWE-20 (0.22) and CWE-787 (0.26) highlights the model's challenge in accurately classifying these vulnerabilities, possibly due to the complexity of the patterns associated with these CWEs or a limited representation in the training data.

The aggregated accuracy of 0.92 and the weighted average precision, recall, and F1 score reflect the model's high competence in vulnerability classification across diverse vulnerabilities. However, the varying performance across different CWEs underscores the importance of continued model refinement and targeted training to enhance the model's sensitivity and specificity, particularly for those vulnerabilities where it currently underperforms. The confusion matrix for multiclass classification on the FalconVulnDB Dataset is shown in Fig. 7.

As evident in the matrix, the model demonstrates strong performance in detecting the "Not-Vulnerable" class, with a high number of true positives and minimal misclassifications. However, for certain challenging vulnerability types, such as CWE-20 (Improper Input Validation) and CWE-787 (Out-of-Bounds Write), the model struggles to achieve the same level of precision and recall. This is likely due to the limited support for these classes (440 and 524 samples, respectively) in the dataset, leading to a higher rate of misclassification into more frequent categories, such as CWE-119 (Memory Corruption) or the "Other" class.

Furthermore, rare and complex classes like CWE-476 (NULL Pointer Dereference) and CWE-190 (Integer Overflow or Wraparound) show lower performance compared to well-represented classes, such as CWE-78 (OS Command Injection) and CWE-121 (Stack-Based Buffer Overflow), which have higher support and more distinct patterns. The confusion matrix highlights these discrepancies, offering valuable insights into the model's misclassification patterns. For future work, we plan to enhance the dataset by augmenting samples for underrepresented classes and leveraging advanced techniques such as the focal loss [64] to prioritize learning on these challenging categories. These improvements will help address the current limitations and improve the model's robustness for detecting rare vulnerabilities.

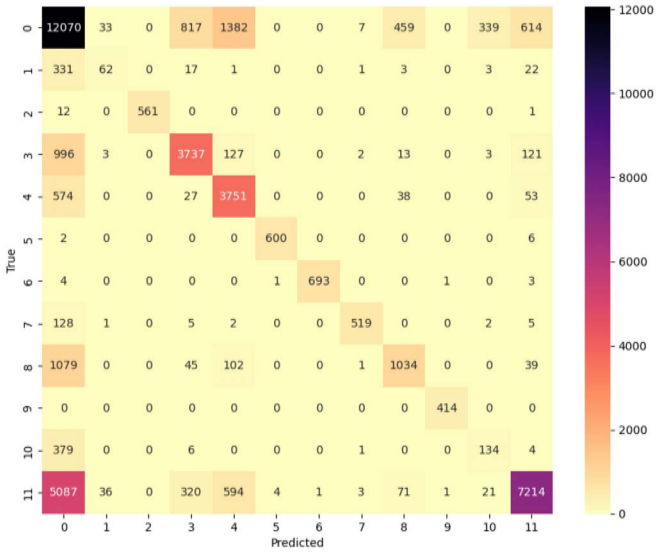


Fig. 7. Confusion Matrix of `SecureFalcon` 121M Multi-classification using FalconVulnDB dataset.

TABLE XII
COMPREHENSIVE EVALUATION METRICS FOR SECUREFALCON 121M WITH LR = 2E-5 USING THE FALCONVULNDB DATASET

Metric	**Value**
Cohen's Kappa	0.86
MCC (Matthews Corr. Coeff.)	0.79
Macro F1-Score	0.74
Weighted F1-Score	0.90
ROC-AUC (Macro Avg)	0.88
PR-AUC (Macro Avg)	0.85
Accuracy	0.92
Macro Precision	0.79
Macro Recall	0.72
Weighted Precision	0.91
Weighted Recall	0.91
Specificity (Macro Avg)	0.89
Log Loss	0.35
Brier Score	0.15
Hamming Loss	0.08

C. Comprehensive Evaluation Metrics

Table XII presents a comprehensive evaluation of the `SecureFalcon` 121M model trained with a learning rate of 2×10^{-5} on the FalconVulnDB dataset, showcasing robust performance across multiple metrics. The following metrics are used in our analysis:

- Cohen's Kappa: Measures the agreement between the predicted and actual classifications, accounting for the possibility of agreement occurring by chance.

$$\kappa = \frac{P_o - P_e}{1 - P_e} \tag{11}$$

  where P_o is the observed agreement and P_e is the expected agreement by chance.
- MCC (Matthews Correlation Coefficient): Evaluates the quality of binary classifications, considering true and false

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

positives and negatives.

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \tag{12}$$

where TP (True Positive) and TN (True Negative) represent the correctly predicted positive and negative instances, respectively, while FP (False Positive) and FN (False Negative) denote the incorrectly predicted positive and negative instances.

- Macro F1-Score: The unweighted average of F1-scores for each class, treating all classes equally.

$$\text{F1}_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C}\frac{2 \times P_i \times R_i}{P_i + R_i} \tag{13}$$

where C is the number of classes, P_i is the precision, and R_i is the recall for class i.

- Weighted F1-Score: The average of F1-scores weighted by the number of true instances for each class.

$$\text{F1}_{\text{weighted}} = \sum_{i=1}^{C} w_i \times \text{F1}_i \tag{14}$$

where $w_i = \frac{\text{Support}_i}{\sum_{j=1}^{C}\text{Support}_j}$.

- ROC-AUC (Macro Avg): The average Area Under the Receiver Operating Characteristic Curve across all classes, assessing the model's ability to distinguish between classes.

$$\text{ROC-AUC}_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C}\text{ROC-AUC}_i \tag{15}$$

- PR-AUC (Macro Avg): The average Area Under the Precision-Recall Curve across all classes, focusing on the trade-off between precision and recall.

$$\text{PR-AUC}_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C}\text{PR-AUC}_i \tag{16}$$

- Macro Precision: The unweighted average precision across all classes, emphasizing the model's ability to correctly identify positive instances.

$$\text{Precision}_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C} P_i \tag{17}$$

- Macro Recall: The unweighted average recall across all classes, highlighting the model's ability to capture all relevant instances.

$$\text{Recall}_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C} R_i \tag{18}$$

- Weighted Precision: The precision score weighted by the number of true instances for each class, reflecting overall precision performance.

$$\text{Precision}_{\text{weighted}} = \sum_{i=1}^{C} w_i \times P_i \tag{19}$$

- Weighted Recall: The recall score weighted by the number of true instances for each class, indicating overall recall performance.

$$\text{Recall}_{\text{weighted}} = \sum_{i=1}^{C} w_i \times R_i \tag{20}$$

- Specificity (Macro Avg): The average specificity across all classes, measuring the model's ability to correctly identify negative instances.

$$\text{Specificity}_{\text{macro}} = \frac{1}{C}\sum_{i=1}^{C}\frac{TN_i}{TN_i + FP_i} \tag{21}$$

- Log Loss: Evaluates the uncertainty of the model's predictions based on the probability estimates.

$$\text{Log Loss} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)\right] \tag{22}$$

where N is the number of samples, y_i is the true label, and p_i is the predicted probability.

- Brier Score: Measures the mean squared difference between predicted probabilities and actual outcomes.

$$\text{Brier Score} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2 \tag{23}$$

- Hamming Loss: The fraction of incorrect labels to the total number of labels.

$$\text{Hamming Loss} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{1}(y_i \neq \hat{y}_i) \tag{24}$$

A high Accuracy of 0.92 and a Weighted F1-Score of 0.90 suggest strong overall effectiveness and good handling of class imbalances. Cohen's Kappa of 0.86 and MCC of 0.79 underscore the reliability and stability of the model's predictions. At the same time, the ROC-AUC (Macro Avg) of 0.88 and PR-AUC (Macro Avg) of 0.85 demonstrate its discriminative power and capacity to maintain solid precision and recall across classes.

The differences between the macro- and weighted-average metrics (e.g., Macro F1-Score of 0.74 versus Weighted F1-Score of 0.90) highlight that certain less frequent classes may not be identified as accurately as more common ones. The Log Loss of 0.35 and Brier Score of 0.15 indicate relatively good calibration, suggesting that the model's probability estimates are reasonably aligned with observed outcomes. Additionally, a Hamming Loss of 0.08 and a Specificity (Macro Avg) of 0.89 confirm that the model makes relatively few incorrect predictions and can identify negative instances correctly. These findings suggest that, while *SecureFalcon* performs admirably, further improvements—such as targeted data augmentation or calibration adjustments—could enhance its robustness across all vulnerability categories.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
COMPARISON OF SECUREFALCON WITH LLM MODELS AND TRADITIONAL MACHINE LEARNING USING THE FORMAI AND FALCONVULNDB DATASETS

Task	ML type	Model	FormAI dataset				FalconVulnDB dataset			
			Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Multiclass	LLM models	RoBERTa	0.70	0.72	0.68	0.70	0.85	0.87	0.84	0.85
		BERT	0.70	0.71	0.69	0.70	0.85	0.86	0.84	0.85
		CodeBERT	0.71	0.73	0.70	0.71	0.88	0.89	0.87	0.88
		SecureFalcon	**0.92**	0.92	0.93	0.93	**0.92**	0.91	0.91	0.90
	Traditional ML	KNN	0.76	0.75	0.76	0.76	0.80	0.79	0.80	0.80
		LR	0.73	0.74	0.72	0.73	0.77	0.78	0.76	0.77
		NB	0.64	0.66	0.62	0.64	0.68	0.69	0.67	0.68
		SVM	0.73	0.73	0.72	0.73	0.77	0.78	0.77	0.77
		RF	0.77	0.78	0.76	0.77	0.81	0.81	0.80	0.81
		DT	0.72	0.72	0.71	0.72	0.60	0.61	0.59	0.60
		LDA	0.73	0.74	0.73	0.74	0.77	0.78	0.76	0.77
Binary	LLM models	RoBERTa	0.82	0.83	0.81	0.82	0.86	0.87	0.85	0.86
		BERT	0.81	0.82	0.80	0.81	0.87	0.88	0.86	0.87
		CodeBERT	0.83	0.84	0.82	0.83	0.89	0.90	0.88	0.89
		SecureFalcon	**0.94**	0.94	0.94	0.94	**0.92**	0.93	0.91	0.92
	Traditional ML	KNN	0.76	0.77	0.75	0.76	0.86	0.86	0.85	0.86
		LR	0.86	0.87	0.86	0.86	0.86	0.87	0.85	0.86
		NB	0.76	0.77	0.74	0.75	0.81	0.82	0.80	0.81
		SVM	0.86	0.87	0.85	0.86	0.89	0.90	0.88	0.89
		RF	0.91	0.91	0.90	0.91	0.89	0.90	0.88	0.89
		DT	0.87	0.88	0.85	0.86	0.86	0.87	0.84	0.85
		LDA	0.86	0.86	0.85	0.85	0.85	0.85	0.85	0.85

ML: Machine learning, KNN: k-Nearest Neighbors, LR: Logistic Regression, NB: Naive Bayes, SVM: Support Vector Machine, RF: Random Forest, DT: Decision Tree, and LDA: Linear Discriminant Analysis.

D. Comparison of SecureFalcon With LLM, ML, and DL Models

Table XIII compares various machine learning (ML) models and their accuracy in performing multiclass and binary classification tasks on the FormAI and FalconVulnDB datasets. Large Language Models (LLMs) such as RoBERTa, BERT, CodeBERT, and `SecureFalcon` show a notable performance difference compared to traditional ML models for multiclass classification. `SecureFalcon`, in particular, stands out with the highest accuracy of 0.92 on both datasets, significantly outperforming other LLM models like RoBERTa and BERT, which achieve accuracies of around 0.70-0.85. Among the traditional ML models, Random Forest (RF) performs the best with accuracies of 0.77 on the FormAI dataset and 0.81 on the FalconVulnDB dataset. However, it still lags behind the LLM models.

The trend remains similar in binary classification, with `SecureFalcon` leading with an accuracy of 0.94 on the FormAI dataset and 0.92 on the FalconVulnDB dataset. Other LLM models, such as RoBERTa, BERT, and CodeBERT, also perform well, achieving accuracies between 0.81 and 0.89 across both datasets. Traditional ML models show competitive results in this task, with Random Forest achieving an accuracy of 0.91 on the FormAI dataset and 0.89 on the FalconVulnDB dataset, which is relatively close to the performance of some LLMs. However, the consistently superior performance of `SecureFalcon` highlights the advantages of fine-tuning large models specifically for software vulnerability detection tasks, making it a robust choice for both multiclass and binary classification in cybersecurity contexts.

Can traditional machine learning models effectively capture vulnerability patterns without sophisticated semantic understanding? For the FormAI dataset, which is synthetic and constructed in a manner favoring simpler patterns, the answer appears to be yes. In the multiclass scenario on the FormAI dataset, CodeBERT—a general pre-trained LLM fine-tuned for classification—achieves an accuracy of 0.71, while specific traditional ML models, such as Random Forest (RF), reach up to 0.77. This suggests that vulnerability patterns in FormAI can be successfully modeled using classical feature engineering or straightforward pattern recognition. Traditional ML methods, often relying on syntactic features (e.g., token frequencies or specific keywords), perform competitively because well-established, surface-level indicators may represent the dataset's vulnerabilities.

But what happens when the complexity of vulnerabilities increases, demanding deeper semantic comprehension? In the FalconVulnDB dataset, where vulnerabilities involve more subtle and context-dependent cues, traditional ML models show a performance plateau (e.g., RF at 0.81 accuracy in multiclass classification). In contrast, LLM-based models—pre-trained on extensive corpora and adept at contextualizing code semantics—excel (e.g., CodeBERT at 0.88). The heightened complexity and nuance of FalconVulnDB vulnerabilities seemingly require the richer representational capacity and more advanced reasoning capabilities that LLMs provide.

Table XVI compares our proposed `SecureFalcon` model against several state-of-the-art deep learning-based vulnerability detection methods, including SySeVR [41], VulDeePecker [6], and Devign [42]. While the earlier approaches often rely on hand-crafted features (e.g., syntax characteristics or code

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV
COMPARISON OF SECUREFALCON WITH C/C++ VULNERABILITY DETECTION TOOLS

Tool/Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	Inference Time (ms/sample)
SecureFalcon	93.5	92.0	91.8	91.9	500
Cppcheck	81.0	79.5	80.2	79.8	150
Clang Static Analyzer	85.2	83.0	84.0	83.5	200
ESBMC	90.0	91.0	87.5	89.2	2000

gadgets) and are typically limited to detecting a narrower set of vulnerabilities, `SecureFalcon` leverages a large language model architecture derived from Falcon-40B. This allows `SecureFalcon` to incorporate both synthetically generated data and real-world vulnerability instances drawn from a broad set of sources (FalconVulnDB), ultimately enabling it to target a far more comprehensive range of CWE types, including the industry-recognized Top 25 Most Dangerous CWEs[11]. In contrast, the earlier methods examined focus on a smaller subset of vulnerabilities and do not integrate synthetic data to enhance coverage and robustness.

E. Comparison of SECUREFALCON With C/C++ Vulnerability Detection Tools

Table XIV presents a comparative evaluation of `SecureFalcon`, Cppcheck[12], Clang Static Analyzer[13], and ESBMC[14]. We randomly sampled 50 code segments from the test part of our FalconVulnDB dataset and manually validated each prediction. For the inference of `SecureFalcon`, we used an NVIDIA A100 GPU with 40 GB of memory. While static analyzers like Cppcheck and Clang Static Analyzer excel in speed and are easy to integrate, they often miss context-dependent vulnerabilities. ESBMC is a mature bounded model checker that supports the verification of single- and multi-threaded C/C++ programs, but it has limitations: it is both slower and requires compilable code. Some code snippets in the dataset—particularly those that are not fully compilable—could not be analyzed by ESBMC, thereby reducing its practical coverage. In contrast, `SecureFalcon`'s LLM-based approach not only offers strong detection rates and nuanced semantic understanding but can also handle snippets that do not compile, expanding its applicability. Although `SecureFalcon`'s inference time is longer than lightweight tools, the accuracy gains and flexibility in handling diverse code samples justify the trade-off in scenarios demanding thorough security checks.

Ultimately, while traditional static analyzers are suitable for quick preliminary scans, ESBMC is valuable for in-depth formal verification of compilable code. `SecureFalcon` is a versatile solution that effectively balances accuracy, context awareness, scalability, and broader code coverage. Furthermore, combining traditional static analyzers and ESBMC with LLMs like `SecureFalcon` can provide synergistic benefits, leveraging the speed and simplicity of static analyzers, the rigorous verification of formal tools, and the deep contextual understanding of LLMs. This integrated approach ensures comprehensive vulnerability detection, maximizing both efficiency and accuracy across various development and deployment scenarios.

F. Inference Time of `SecureFalcon`

Inference performance is a critical metric for evaluating the applicability of machine learning models in real-world scenarios, particularly for tasks such as vulnerability classification in C/C++ codebases.`SecureFalcon`, with its compact architecture of 121 million parameters, demonstrates exceptional efficiency with a *Time to First Token (TTFT)* of 0.3 seconds and a total classification time of 0.6 seconds for typical input sizes of 300-500 tokens. It achieves a high throughput of 45-50 requests per second (RPS) and processes 95 tokens per second on a single NVIDIA A100 GPU. Furthermore, `SecureFalcon` is optimized for CPU inference, achieving competitive latency and throughput on hardware such as the AMD Ryzen 7 7840U CPU (8 cores, 16 threads, 30W TDP) with 32GB LPDDR5X RAM. These results highlight `SecureFalcon`'s suitability for edge devices and on-premises deployments, offering the speed and scalability required for enterprise-scale vulnerability detection. By delivering low-latency and high-throughput inference across a range of hardware configurations, `SecureFalcon` enables seamless integration into real-time workflows, including CI/CD pipelines, and supports efficient analysis of large-scale C/C++ codebases, making it an optimal choice for modern software security applications.

G. Ablation Studies

While our main experiments demonstrate that `SecureFalcon` achieves strong performance in detecting software vulnerabilities, it is crucial to understand the contribution of individual model components and training configurations. To this end, we performed ablation studies by systematically modifying or removing certain model aspects and evaluating their impact on accuracy and F1-score. The results, summarized in Table XV, provide insights into which factors most influence the model's performance.

We started with the baseline model configuration described in Table IV, which includes positional embeddings, special token embeddings, twelve attention heads, and a dropout probability of 0.1 for both attention and hidden layers. From this baseline, we disabled positional embeddings, observing a noticeable drop in accuracy and F1 score. This indicates that positional information is essential in capturing code structure and identifying vulnerability patterns.

[11]https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html
[12]https://cppcheck.sourceforge.io/
[13]https://clang-analyzer.llvm.org/
[14]https://github.com/esbmc/esbmc

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

TABLE XV
ABLATION RESULTS SHOWING THE IMPACT OF REMOVING OR MODIFYING KEY COMPONENTS AND CONFIGURATIONS OF SECUREFALCON. EACH SETTING WAS EVALUATED USING THE BINARY CLASSIFICATION TASK ON THE FORMAI DATASET (LR = 2E-5). THE BASELINE CONFIGURATION REFERS TO ALL FEATURES AND PARAMETERS AS LISTED IN TABLE IV

Configuration	Description of Change	Accuracy	F1-score
Baseline (Full Model)	All components enabled: positional encodings (514 max), special token embeddings, 12 attention heads, dropout=0.1	0.94	0.94
No Positional Embeddings	Removed positional embeddings from the transformer layers, disabling positional indexing of tokens	0.91 (-0.03)	0.90 (-0.04)
No Special Token Embeddings	Excluded the domain-specific tokens (keywords, punctuation, API calls) from the vocabulary	0.89 (-0.05)	0.91 (-0.03)
Fewer Attention Heads	Reduced number of attention heads from 12 to 6, decreasing model's representational capacity	0.90 (-0.04)	0.90 (-0.04)
Increased Dropout	Doubled attention and hidden dropout from 0.1 to 0.2, increasing regularization	0.92 (-0.02)	0.93 (-0.01)

TABLE XVI
COMPARISON OF SECUREFALCON WITH DEEP LEARNING-BASED VULNERABILITY DETECTION METHODS

Model	Method	Datasets Used	Synthetic Data	Real Data	CWE Types Targeted	Top 25 CWEs
SySeVR [41]	Syntax & semantics-based vector representations	4 kinds of vulnerability syntax characteristics	No	Yes	Limited (fewer vulnerabilities)	No
VulDeePecker [6]	Code gadgets & deep learning classification	Curated dataset from known software	No	Yes	Limited (e.g., buffer overflow)	No
Devign [42]	Graph neural networks on code semantics	4 large-scale open-source C projects	No	Yes	Limited (depends on projects)	No
SecureFalcon	LLM-based (derived from Falcon-40B)	FalconVulnDB (combining multiple datasets)	Yes	Yes	Broad (Top 25 + beyond)	Yes

Removing the special token embeddings—derived from programming language keywords, punctuation, and API calls—led to a notable decrease in accuracy and a slight reduction in the F1 score. These domain-specific tokens provide valuable lexical cues, enabling the model to recognize subtle vulnerability signatures that might otherwise be missed.

Similarly, reducing the number of attention heads from twelve to six diminished the model's performance, suggesting that the rich representational capacity provided by multiple attention heads is necessary for practical reasoning about complex code patterns. Increasing the dropout rate from 0.1 to 0.2 did not improve results beyond the baseline, only modestly reduced performance. This indicates that while regularization helps prevent overfitting, other factors—such as positional information and specialized tokens—are more critical for overall accuracy and robustness.

In summary, the ablation results highlight the importance of maintaining positional embeddings, preserving domain-specific token embeddings, and ensuring sufficient representational capacity through multiple attention heads.

H. Limitations and Future Works

`SecureFalcon` is currently limited in identifying zero-day vulnerabilities because it relies on known vulnerability patterns and cannot detect previously unseen threats. Additionally, being trained exclusively on C/C++ datasets restricts its ability to analyze other programming languages and prevents it from generating detailed reports that explain the root causes of vulnerabilities within the code. As future work, `SecureFalcon` will integrate Agentic Retrieval-Augmented Generation (RAG) to address these challenges, enabling AI-driven agents to investigate and exploit potential zero-day vulnerabilities autonomously. To uncover previously unknown vulnerabilities, these agents will leverage diverse data sources, including proprietary databases, real-time threat intelligence, and specialized security tools. Expanding `SecureFalcon's` training to encompass multiple programming languages will enhance its versatility and effectiveness across various software environments. Using Agentic RAG, `SecureFalcon` can dynamically adapt its detection methods, ensuring a more proactive and comprehensive approach to identifying and mitigating emerging security threats.

In addition to overcoming zero-day vulnerability detection and expanding language support, `SecureFalcon` aims to enhance its capabilities by transitioning from classification to generation tasks. This evolution will enable `SecureFalcon` to provide developers with more detailed and actionable insights. Enhancements include generating comprehensive reports that describe detected vulnerabilities, highlighting affected code snippets, and analyzing potential impacts. Furthermore, `SecureFalcon` will outline steps to reproduce issues, including environment setup, input data, and execution steps, and offer remediation guidance with fix recommendations and secure coding practices. By fine-tuning `SecureFalcon` for generation tasks and integrating these features with development tools through Agentic RAG, `SecureFalcon` will ensure real-time feedback and automated code reviews.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

VII. Conclusion

Our study highlights the significant potential of LLMs in detecting software vulnerabilities, especially in cybersecurity. By fine-tuning *FalconLLM*, we developed *SecureFalcon*, a novel model capable of distinguishing between vulnerable and non-vulnerable C/C++ code samples. Tested on different datasets from the literature, our model achieved a remarkable 94% accuracy rate in binary classification and 92% in multiclassification, highlighting its efficacy. Furthermore, we accomplish these results by utilizing a relatively small set of parameters, totaling 121 million, within the `SecureFalcon` model. We believe further advancements will enhance such models' capabilities, expand with distinct vulnerability types, and extend to other programming languages to strengthen software system security and foster a more secure digital realm. A promising extension of this work would include an automated self-healing system that identifies and remedies the vulnerabilities. Such a system could make software more resilient against potential threats, thus enhancing overall cybersecurity. A fast inference time model like `SecureFalcon` has the potential to be highly effective in automated software completion frameworks. We plan to continue our research in this direction.

Acknowledgment

The authors would like to thank the Technology Innovation Institute for providing the computing resources necessary for training the models used in this research. This work would not have been possible without the institute's support.

References

[1] D. J. Solove, *The Digital Person: Technology and Privacy in the Information Age*, vol. 1. New York, NY, USA: NYU Press, 2004.

[2] M. Abdel-Rahman et al., "Advanced cybersecurity measures in it service operations and their crucial role in safeguarding enterprise data in a connected world," *Eigenpub Rev. Sci. Technol.*, vol. 7, no. 1, pp. 138–158, 2023.

[3] Z. Li, Z. Liu, W. K. Wong, P. Ma, and S. Wang, "Evaluating C/C++ vulnerability detectability of query-based static application security testing tools," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4600–4618, Sep./Oct. 2024.

[4] Y. Sun et al., "LLM4Vuln: A unified evaluation framework for decoupling and enhancing LLMs' vulnerability reasoning," 2024, *arXiv:2401.16185*.

[5] R. Jain, N. Gervasoni, M. Ndhlovu, and S. Rawat, "A code centric evaluation of C/C++ vulnerability datasets for deep learning based vulnerability detection techniques," in *Proc. 16th Innov. Softw. Eng. Conf.*, 2023, pp. 1–10.

[6] Z. Li et al., "VulDeePecker: A deep learning-based system for vulnerability detection," 2018, *arXiv:abs/1801.01681*.

[7] N. Ziems and S. Wu, "Security vulnerability detection using deep learning natural language processing," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2021, pp. 1–6.

[8] X. Duan et al., "VulSniper: Focus your attention to shoot fine-grained vulnerabilities," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, Menlo Park, CA, USA: AAAI Press, 2019, pp. 4665–4671.

[9] H. Hanif and S. Maffeis, "VulBERTa: Simplified source code pre-training for vulnerability detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE, 2022, pp. 1–8.

[10] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.

[11] P. E. Black, "A software assurance reference dataset: thousands of programs with known bugs," *J. Res. Nat. Inst. Standards Technol.*, vol. 123, pp. 1–3, Apr. 2018. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7339570/

[12] F. E. Boland Jr. and P. E. Black, "The Juliet 1.1 C/C++ and Java test suite," *NIST*, vol. 45, no. 10, pp. 88–90, Oct. 2012. Accessed: Oct. 12, 2021. [Online]. Available: https://www.nist.gov/publications/juliet-11-cc-and-java-test-suite

[13] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, *Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks*. Red Hook, NY, USA: Curran Associates Inc., 2019, pp. 10197–10207.

[14] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Softw. Repositories (MSR)*, New York, NY, USA: ACM, Sep. 2020, pp. 508–512. [Online]. Available: https://doi.org/10.1145/3379597.3387501

[15] L. Kim and R. Russell, "Draper VDISC dataset—Vulnerability detection in source code," OSF, 2018. [Online]. Available: https://osf.io/d45bw/

[16] Y. Chen, Z. Ding, X. Chen, and D. Wagner, "DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection," Apr. 2023. [Online]. Available: http://arxiv.org/abs/2304.00409

[17] A. Habib and M. Pradel, "How many of all bugs do we find? A study of static bug detectors," in *Proc. 33rd ACM/IEEE Int. Conf. Autom. Softw. Eng.*, 2018, pp. 317–328.

[18] J. Park, I. Lim, and S. Ryu, "Battles with false positives in static analysis of JavaScript web applications in the wild," in *Proc. 38th Int. Conf. Softw. Eng. Companion*, 2016, pp. 61–70.

[19] V. Hartonas-Garmhausen, S. V. A. Campos, A. Cimatti, E. M. Clarke, and F. Giunchiglia, "Verification of a safety-critical railway interlocking system with real-time constraints," *Sci. Comput. Program.*, vol. 36, no. 1, pp. 53–64, 2000. [Online]. Available: https://doi.org/10.1016/S0167-6423(99)00016-7

[20] C. Heitmeyer, J. Kirby, B. Labaw, M. Archer, and R. Bharadwaj, "Using abstraction and model checking to detect safety violations in requirements specifications," *IEEE Trans. Softw. Eng.*, vol. 24, no. 11, pp. 927–948, Nov. 1998.

[21] J. Lahtinen, J. Valkonen, K. Björkman, J. Frits, I. Niemelä, and K. Heljanko, "Model checking of safety-critical software in the nuclear engineering domain," *Rel. Eng. Syst. Saf.*, vol. 105, pp. 104–113, 2012.

[22] A. Biere, A. Cimatti, E. M. Clarke, O. Strichman, and Y. Zhu, "Bounded model checking," in *Handbook of Satisfiability*, vol. 185, no. 99, Amsterdam, The Netherlands: IOS Press, 2009, pp. 457–481.

[23] R. S. Menezes et al., "ESBMC v7.4: Harnessing the power of intervals (competition contribution)," in *Proc. Tools Algorithms Construction Anal. Syst. (TACAS)*, LNCS, vol. 14572, Berlin, Germany: Springer, 2024, pp. 376–380.

[24] N. Nehorai, "Analyzing common vulnerabilities introduced by code-generative AI," Hackernoon, 2024. [Online]. Available: https://hackernoon.com/analyzing-common-vulnerabilities-introduced-by-code-generative-ai

[25] E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., *Handbook of Model Checking*. Berlin, Germany: Springer, 2018.

[26] A. M. Turing, "On computable numbers, with an application to the entscheidungsproblem," *Proc. London Math. Soc.*, vol. s2–42, no. 1, 1936, pp. 230–265.

[27] M. Sipser, *Introduction to the Theory of Computation*. Boston, MA, USA: Cengage Learning, 2012.

[28] N. Tihanyi, T. Bisztray, R. Jain, M. A. Ferrag, L. C. Cordeiro, and V. Mavroeidis, "The formAI dataset: Generative AI in software security through the lens of formal verification," in *Proc. 19th Int. Conf. Predictive Models Data Analytics Softw. Eng.*, 2023, pp. 33–43.

[29] H. Hanif, M. H. N. M. Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar, "The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches," *J. Netw. Comput. Appl.*, vol. 179, 2021, Art. no. 103009.

[30] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Reading, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2006.

[31] H. Barbosa et al., "cvc5: A versatile and industrial-strength SMT solver," in *Proc. 28th Int. Conf. Tools Algorithms Construction Anal. Syst. (TACAS), Held as Part Eur. Joint Conf. Theory Pract. Softw. (ETAPS)*, Munich, Germany, Apr. 2–7, 2022, *Proc., Part I*, Lecture Notes Comput. Sci., D. Fisman and G. Rosu, Eds., vol. 13243. Berlin, Germany: Springer, 2022, pp. 415–442. [Online]. Available: https://doi.org/10.1007/978-3-030-99524-9_24

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.

[32] A. Niemetz and M. Preiner, "Bitwuzla," in *Proc. 35th Int. Conf. Comput. Aided Verification (CAV)*, Paris, France, Part II, Lecture Notes Comput. Sci., C. Enea and A. Lal, Eds., vol. 13965. Berlin, Germany: Springer, Jul. 17–22, 2023, pp. 3–17. [Online]. Available: https://doi.org/10.1007/978-3-031-37703-7_1

[33] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and J. Rehof, Eds. Berlin, Heidelberg: Springer, 2008, pp. 337–340.

[34] T. Ahmed and P. Devanbu, "Few-shot training LLMs for project-specific code-summarization," in *Proc. 37th IEEE/ACM Int. Conf. Autom. Softw. Eng.*, 2022, pp. 1–5.

[35] A. Tarassow, "The potential of LLMs for coding with low-resource and domain-specific programming languages," 2023, *arXiv:2307.13018*.

[36] X. Cheng, G. Zhang, H. Wang, and Y. Sui, "Path-sensitive code embedding via contrastive learning for software vulnerability detection," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 519–531.

[37] Z. Chen, S. Kommrusch, and M. Monperrus, "Neural transfer learning for repairing security vulnerabilities in C code," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, pp. 147–165, Jan. 2023.

[38] J. Yin, M. Tang, J. Cao, and H. Wang, "Apply transfer learning to cybersecurity: Predicting exploitability of vulnerabilities by description," *Knowl.-Based Syst.*, vol. 210, 2020, Art. no. 106529.

[39] P. Mahbub, O. Shuvo, and M. M. Rahman, "Explaining software bugs leveraging code structures in neural machine translation," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE, 2023, pp. 640–652.

[40] N. Ito, M. Hashimoto, and A. Otsuka, "Feature extraction methods for binary code similarity detection using neural machine translation models," *IEEE Access*, vol. 11, pp. 102796–102805, 2023.

[41] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, Jul./Aug. 2022.

[42] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, *Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[43] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[44] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[45] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[46] J. Zhou et al., "Finding a needle in a haystack: Automated mining of silent vulnerability fixes," in *Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE, 2021, pp. 705–716.

[47] K. Singh, S. S. Grover, and R. K. Kumar, "Cyber security vulnerability detection using natural language processing," in *Proc. IEEE World AI IoT Congr. (AIIoT)*, Piscataway, NJ, USA: IEEE, 2022, pp. 174–178.

[48] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[49] S. Kim, S. Woo, H. Lee, and H. Oh, "VUDDY: A scalable approach for vulnerable code clone discovery," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2017, pp. 595–614.

[50] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, and J. Hu, "VulPecker: An automated vulnerability detection system based on code similarity analysis," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl. (ACSAC)*, New York, NY, USA: ACM, 2016, pp. 201–213. [Online]. Available: https://doi.org/10.1145/2991079.2991102

[51] P. Zeng, G. Lin, L. Pan, Y. Tai, and J. Zhang, "Software vulnerability analysis and discovery using deep learning techniques: A survey," *IEEE Access*, vol. 8, pp. 197158–197172, 2020.

[52] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.

[53] E. Almazrouei et al., "The falcon series of open language models," 2023. [Online]. Available: https://arxiv.org/abs/2311.16867

[54] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.

[55] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 3111–3119.

[56] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[57] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.

[58] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," 2021, *arXiv:2104.09864*.

[59] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[60] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2016, *arXiv:1606.08415*.

[61] C. M. Bishop and N. M. Nasrabadi, *Pattern Recognition and Machine Learning*. Berlin, Germany: Springer, 2006, vol. 4, no. 4.

[62] M. Y. R. Gadelha, F. R. Monteiro, J. Morse, L. C. Cordeiro, B. Fischer, and D. A. Nicole, "ESBMC 5.0: An industrial-strength C model checker," in *Proc. ASE,* New York, NY, USA: ACM, 2018, pp. 888–891.

[63] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. 19th Int. Conf. Mining Softw. Repositories*, 2022, pp. 608–620.

[64] J. Mukhoti, V. Kulharia, A. Sanyal, S. Golodetz, P. Torr, and P. Dokania, "Calibrating deep neural networks using focal loss," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 15288–15299.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:12:07 UTC from IEEE Xplore. Restrictions apply.